level data access technology called ADO.NET. It exposes data to the application in structures that are similar to in-memory database tables or views. If you want to access pieces of data, you have to use indexers and conversion or casting:

```
var myInt = Convert.ToInt32(dataTable.Rows[1][4]);
```

This is an ugly and error-prone way to develop an application. All it needs to fail is someone to change the order of the columns in an SQL statement that the previous C# statement depends on, because the materialized value in the targeted location can no longer be converted to an `int`. These days, developers generally prefer to work with data as objects (e.g., a `Book` class or an `Author` class) and will use an object-relational mapping (ORM) tool to manage the communication between the database and the application. The ORM also takes care of (among other things) mapping data from a database query to a specified object or collection of objects for you.

Several ORM tools are available to .NET developers. Most of them are owned and managed by third parties. The ORM I have selected for this book is Entity Framework Core (EF Core). I will be using this ORM because it is a Microsoft technology and part of .NET. Figure 1.7 is an updated version of figure 1.3 and shows where EF Core fits within the .NET stack.